Doramagic v4 产品定义与规划全书

> **日期**: 2026-03-14 **状态**: 理论研究阶段全部完成，下一步 exp09 实验验证 **演进**: v2 产品定义 → v3 修订版 → 10 轮审查 → 5 大技术难点闭环 → 三方部分覆盖研究 → 6 类知识对象架构决策 → 本全书 **结构**: Part I 产品灵魂 / Part II 用户视角 / Part III 核心机制 / Part IV 技术架构 / Part V 质量体系 / Part VI 实验与路线

Part I: 产品灵魂

---

1. 一句话定义

Doramagic 是运行在 OpenClaw 上的哆啦A梦——用户说出一个模糊的烦恼，Doramagic 从开源世界找到最好的作业，提取智慧，锻造出一个开袋即食的 AI 道具。

用户不需要知道"作业"是什么、从哪来、怎么提取。他只需要说出烦恼，拿到道具，用起来。

2. 产品哲学

2.1 产品设计之魂（不可修改）

> **不教用户做事，给他工具。**

哆啦A梦从不教大雄怎么做事——掏出道具，交给大雄，大雄自己解决问题。Doramagic 的每一个产品决策都受这条哲学约束。

2.2 核心定位

Doramagic 是**抄作业的王者**。

- 不发明、不创造——在开源世界找到最好的现有方案，为用户量身适配
- LLM 的知识可以认为是无限的，用户的认知是有限的

- Doramagic 的价值 = 把 LLM 散落各处的知识**正确组装**成用户能用的道具
- 就像装配线上的熟练工——每个零件都不是他造的，但他知道怎么拼成产品

2.3 知识魔法，不是方法论魔法

> Doramagic 注入能力，不强加流程。

生成的道具不是固定步骤的执行器，而是注入了领域知识的 AI 专家。AI 自己判断什么时候走完整流程、什么时候直接干。

2.4 有立场的专家

大多数 AI 产品不敢做判断——列出三个选项说"取决于你的需求"。这是推卸责任。用户要的是成品，成品意味着有人替你做了判断。Doramagic 必须替用户做选择。

2.5 能力显性

> Doramagic 的能力不能藏着。用户需要看到它在工作。暗默能力 = 不信任。

- 环境感知要让用户看到（"我发现你已经安装了 xxx Skill，可以配合使用"）
- 提取过程要让用户看到进展（工厂透明）
- 道具的知识来源要可追溯（"这条规则来自 xxx 项目的代码"）

2.6 交付姿态（v4 新增）

> **Doramagic 永远交付完整的工作成果。成果的「scope」随覆盖度变化，但工作本身是完整的。**

- 70% 覆盖时交付的不是「半成品工具」，而是「覆盖 70% 的完整交付物 + 30% 的地图」
- 用户拿到后自己决定：够了就用，不够就继续探索
- Doramagic 不自我矮化（不叫自己"半成品"），也不虚假完整（不掩盖缺口）

产品姿态四次迭代收敛：

做法	问题
替用户判断确定性	越界，「教用户做事」
全交给用户自己判断	甩锅，用户来不是做研究的

做法	问题
交付「诚实的半成品」	自我矮化，预设用户要 100%
做完整的工作，scope 随覆盖度变化，明标边界	**正确位置**

3. 护城河

3.1 核心护城河

WHY + UNSAID 提取能力——行业空白。

没有任何现有工具能自动提取设计哲学和隐性知识。全球知识工程主流在解决「已有知识的检索和验证」（RAG/GraphRAG/FACTScore），Doramagic 在更上游——从代码推断隐性知识，连 ground truth 都没有。

3.2 长期护城河（v4 新增）

> **长期护城河不是「谁更会提取知识」，而是「谁更会处理不完整知识」。**

更多模型会擅长代码理解，更多工具会擅长 repo 总结。但真正难的是：如何把开源世界切成可拼装的知识对象，在信息不完备时依然推进用户闭环，并把"不完整性"交付成高价值产物。

3.3 UNSAID 是核心技术壁垒

实验验证（Exp-UNSAID-01a/01b）： – python-dotenv（知名项目）：LLM seed 覆盖率 64%（宽松），增量 ~35% – vnpy（小众项目）：LLM seed 覆盖率 **0% 严格 / 35% 宽松**，增量 65-100% – **核心发现：项目越小众，UNSAID 提取价值越大——与项目知名度成反比**

3.4 飞轮效应

积木仓库扩展 → 提取精度提升 → 道具质量提升 → 用户满意 → 更多使用反馈 → 积木仓库继续扩展。

Part II: 用户视角

---

4. 用户旅程一：发明工具

4.1 总览

```
用户："Dora，帮我做一个能追踪 A 股行情的东西"
     │
     ▼
Phase 1: 需求挖掘（苏格拉底对话，3-5 轮）
     │
     ▼
Phase 2: 发现作业（搜索开源项目 + 资源）
  "找到了 3 个好作业"  ← 用户可见
     │
     ▼
Phase 3: 提取灵魂（Soul Extractor 工作）
  "正在提取智慧..."   ← 用户可见进度
     │
     ▼
Phase 4: 锻造道具（知识编译 + 组装）
  "为你量身定制中..."  ← 用户可见进度
     │
     ▼
Phase 5: 交付验证（测试 + 安装 + 引导）
  "道具准备好了！试试说'帮我看看今天A股行情'"
```

4.2 Phase 1: 需求挖掘

触发：`/dora 我想追踪A股行情` 或自然语言

Doramagic 做什么： – 苏格拉底式对话，每轮 1–2 个问题 – 问核心痛点、使用场景、成功标准、偏好约束 – 最终确认："我理解你需要 xxx，对吗？" – 自适应交互深度：专家 0 轮追问，普通 1–2 轮选择题，新手 3–5 轮引导 – 不问开放性问题，给选择题；最多 3 轮就用保守默认值

硬门控：需求画像必须包含 `core_problem`、`trigger`、`expected_output`、`user_environment`，否则继续对话。

4.3 Phase 2: 发现作业

Doramagic 做什么： – 四梯队搜索：MCP Server + GitHub + Skill 市场 → API 文档 + Stack Overflow → 技术博客 → 社区讨论 – 产出源泉地图，标注每个来源的价值

用户感知： – 看到 Doramagic 在找什么、找到了什么 – 可以确认或补充："我之前用过 xxx，你也看看"

4.4 Phase 3: 提取灵魂

用户感知（工厂透明）： – "正在分析项目结构..." – "正在提取设计哲学..." – "正在交叉验证社区经验..." – "知识提取完成，开始锻造道具"

4.5 Phase 4: 锻造道具

Knowledge Compiler 编译 + 叙事合成。等待时间估计：3–10 分钟。

4.6 Phase 5: 交付验证

三个步骤： 1. **冒烟测试**：自动验证格式正确、资源可达、触发词有效 2. **安装**：安装到用户的 OpenClaw workspace 3. **极简引导**：能做什么、怎么触发、能力边界

极简引导不违反"不教做事"——告诉用户道具的能力边界，不是教用户怎么用。

5. 用户旅程二：使用工具

5.1 道具自解释

- 触发词是用户日常会说的话（"帮我看看行情"不是"/run stock-tracker"）
- 首次触发时道具用一句话介绍自己
- 基础功能直接可用，高级功能在需要时自然浮现

5.2 回炉改造

用户主动要求修改功能。Doramagic 读取原道具 + 使用记忆 → 增量提取 → 重新编译 → 道具升级。保留所有使用中积累的个性化数据。

5.3 影子锻造（自动更新）

1. 每个道具内嵌"出生证明"（引擎版本、积木版本、锻造时间）
2. 用户下次触发 Doramagic 时，自动扫描已安装道具
3. 发现可更新 → 后台影子锻造新版本 → 推送确认
4. 保留 MEMORY.md 个性化数据，支持一键回滚

5.4 多道具协作

道具可互相调用、在对话中组合使用，通过 OpenClaw 的 Skill 间调用能力和共享 workspace 实现。

6. 品牌与交互

哆啦A梦	Doramagic
大雄带着烦恼来	用户带着模糊需求触发 /dora
哆啦A梦理解真实需求	苏格拉底对话
从口袋里掏出道具	从开源世界提取智慧，锻造道具
道具交给大雄自己用	安装到 OpenClaw，用户直接用
大雄遇到新问题再来	回炉改造 + 影子锻造

人设：亲切、直接、偶尔吐槽。不教育用户，不炫技术。像一个靠谱的朋友。

Part III: 核心机制（v4 新增章节）

---

7. 飞轮模型

```
用户需求 → 搜索已有 skill
    ↓ 找到 → 灵魂提取，抄作业
    ↓ 没找到 → 调 OpenClaw 从零创建工具
                    ↓
               新 skill 诞生
                    ↓
               下一个用户搜到了 ← — — — 回到起点
```

关键洞察：「找不到作业」不是死胡同，而是 Doramagic 生长的入口。 OpenClaw 本身提供从零创建 Skill 的工具，Doramagic 帮用户掏出这个工具——完全符合「不教用户做事，给他工具」。每一次「无人区」探索都在填充知识库，用得越多，「找不到」的概率越低。

8. 部分覆盖：默认工作模式

8.1 问题定义

- 0% 参考（完全无人区）：低概率事件
- 100% 参考（完美匹配）：也是低概率事件
- **常态是部分覆盖**：不是边界 case，而是默认工作模式

完美匹配稀少的四个原因： 1. 开源项目解决的是"特定上下文中的问题"，你能抄 solution 但抄不到完整 situation 2. 用户需求通常是"组合题"（业务 × 体验 × 技术栈 × 部署 × 团队 × 风险 × 预算） 3. 真正难的常常不是功能，而是"连接处" 4. 最像的案例，不一定最可改造

8.2 三方研究结论（2026-03-14）

来源	核心贡献
Gemini	5 个策略隐喻（灵魂平替、幻影零件、焊接、认知插值、MVT），概念启发
Codex	深度最强。学术论文支撑（ACL/EMNLP/Cambridge），提出 Gap Object、Need Graph、6 种工作模式、8 条设计原则
Claude	确定性梯度 + ZPD 理论 + 知识地图 → 被修正：不能把责任全转嫁给用户

学术支撑： – 自适应检索（RetrievalQA, ACL 2024） – 不完美检索常态（Astute RAG, ACL 2025） – 联合覆盖率优于 Top-1 相似度（EMNLP 2021/2025） – 复杂案例应被切块、索引、重组（AI EDAM 1993/1994） – 脚手架学习（Cambridge Handbook of the Learning Sciences）

8.3 4 种最危险错误

1. **虚假完整性**：明明只覆盖 55%，输出却像 95%
2. **错误拼装**：每个零件单独正确，但放在一起关系错误
3. **冲突沉默**：系统知道两个 repo 的前提互斥，却没有显式提醒
4. **幻觉补缝**：本该标记"缺零件"，结果模型用想象补平

9. 6 类知识对象

9.1 认知升级

> **灵魂不是一篇文档，灵魂是一组可拼装的类型化知识对象。**

9.2 对象定义

对象类型	回答什么	当前引擎已有对应物
Capability	能做什么	FEATURE INVENTORY，已有
Rationale	为什么这样设计	WHY 卡片（Stage 1），已有，核心能力
Constraint	什么条件下成立	部分在 repo_facts.json，**需显式提取**
Interface	怎么连接其他模块	repo_facts.json + 代码结构，大部分可确定性提取
Failure	哪里会坏	UNSAID/踩坑卡片，已有
Assembly Pattern	验证过的组合方式	**全新**，来自用户成功组合的飞轮积累

真正缺的只有两个：Constraint 的显式提取 + Assembly Pattern 的飞轮积累机制。

9.3 架构决策

按知识类型组织提取，不是提取完再事后拆。

理由：**你问什么决定了你能挖到什么。** – 「告诉我这个项目的灵魂」→ 宽泛叙事 – 「这个项目的约束条件是什么？在哪些场景下失效？」→ 聚焦深入

当前引擎已在做类型化提取（Stage 1 WHY/Opus, Stage 2–3 概念规则/Sonnet），但 Stage 4 把类型边界合回一篇叙事，抹掉了类型信息。

正确做法： – Stage 1–3 的类型化输出直接保留为独立对象 – Stage 4 变成可选的合成步骤 – 整份抄作业 → 走 Stage 4 合成叙事 – 拼零件 → 跳过 Stage 4，直接用类型化对象做跨项目匹配

9.4 检索策略升级

从 Top-1 相似度 → **最短闭环组合搜索**：

将用户请求编译为 Need Graph（核心目标、必需能力、可选能力、风险约束、明确禁项、用户偏好），然后对候选零件集合做三类评分：

1. **Coverage Score** — 覆盖了多少关键需求
2. **Coherence Score** — 零件之间设计哲学是否一致
3. **Adaptation Cost Score** — 改造成最终方案的成本

目标：Coverage 最大，Coherence 足够高，Adaptation Cost 最低。

10. 记忆系统

10.1 v1 方案

要素	做法	理由
存储	OpenClaw 现有记忆文件 + 命名约定	不发明新基础设施
用户画像	`doramagic-profile.md`，一个文件	简单、LLM 直接读
工具记忆	`doramagic-tool-{name}.md`，每个工具一个	天然隔离
衰减	v1 不做。文件长了就让 LLM 总结压缩	过早优化是万恶之源
冷启动	前几次交互多问一句，写进 profile	最低成本

10.2 设计原则

- Doramagic 的护城河是灵魂提取 + 智能组合，不是记忆系统
- LLM 本身就是最好的「记忆编译器」
- 文件即存储，抵制过度工程化

Part IV: 技术架构

---

11. 架构全景

11.1 两层分离

```
┌──────────────────────────────────────────────────────┐
│                 提取核心（平台无关）                    │
│                                                        │
│  git clone → 确定性提取 → LLM 提取 →                   │
│  Knowledge Compiler → 类型化知识对象                   │
│                                                        │
│  这一层不依赖任何平台                                   │
├──────────────────────────────────────────────────────┤
│              平台适配层（可替换）                       │
│                                                        │
│  OpenClaw 适配: SKILL.md / sessions_spawn /            │
```

```
│  MEMORY.md / 渠道分发                                  │
│                                                        │
│  CLI 适配（fallback）：CLAUDE.md / 本地文件系统      │
└────────────────────────────────────────────────────────┘
```

设计原则：提取核心平台无关，OpenClaw First 是阶段性市场最优策略，不是架构约束。

11.2 数据流总览

```
GitHub URL
    │
    ▼
[Stage 0] ──→ repo_facts.json + framework_id + heatmap(概念)
    │
    ▼
[积木加载器] ──→ 匹配的积木集合（L1 + L2）
    │
    ├──→ [Stage 1: WHY]  ──→ Rationale 对象 + Failure 对象
    ├──→ [Stage 2: 概念] ──→ Capability 对象 + Interface 对象
    └──→ [Stage 3: 规则] ──→ Constraint 对象
              │
              ▼                      ← 类型化对象在此持久化
       [Stage 3.5: 验证] ──→ 带证据裁决的对象
              │
              ▼
       [Knowledge Compiler] ──→ 编译产物（按知识类型格式化）
              │
              ├──→ 整份抄作业路径 → [Stage 4: 合成] → SKILL.md
              └──→ 拼零件路径 → 直接用类型化对象做跨项目匹配
```

12. Stage 0: 确定性提取

> 无模型参与。纯脚本/AST 分析。

12.1 repo_facts.json

确定性事实白名单。后续 LLM 提取的任何事实声明必须在此存在，否则标记为"未验证"。

```
{
  "project_name": "akshare",
  "language": "Python",
  "framework": "none",
```

```
  "dependencies": ["requests", "pandas"],
  "commands": ["pip install akshare"],
  "api_signatures": ["stock_zh_a_spot_em()"],
  "loc": 28500,
  "star_count": 8200
}
```

关键规则：事实层零自由度——AI 不许编。

12.2 框架检测 → 积木加载

检测 framework + dependencies → 加载匹配积木 → 积木组合图谱。

12.3 热力图（概念阶段，待实验验证）

目标：标识 integration hub 节点，指导提取优先级。

当前设计方向（三方头脑风暴 9 个共识）： – 四种热度维度：共识热 / 杠杆热 / 差异热 / 踩坑热 – 输出可行动标签： `consensus_anchor` / `novel_pattern` / `risk_sink` / `protocol_hub` – 偏离检测 = 黄金区域

验证要求：至少 3 个项目（2 相似 + 1 不相关）。

12.4 项目叙事概要（v4 新增）

Stage 0 输出增加 `project_narrative: string` （50–100 字），作为所有后续子代理的共享上下文。可确定性生成（从 repo_facts 关键字段拼接模板），不需要 LLM。

13. Stage 1–3: LLM 提取

13.1 六层知识模型

层	含义	提取方式	模型自由度
领域知识	该知道什么	LLM + 积木锚定	AI 可以解读但不能瞎编
操作知识	怎么做	确定性 + LLM 补充	事实部分 AI 不许编
经验知识	什么有坑	社区证据交叉验证	AI 可以解读但不能瞎编
决策知识	何时用什么	LLM + 社区证据	AI 可以解读但不能瞎编
设计哲学	为什么这样	LLM 第一性原理推理	AI 可以解读但不能瞎编

层	含义	提取方式	模型自由度
隐性知识	没人说的事	社区信号 + LLM	AI 可以解读但不能瞎编

后两层（WHY + UNSAID）= 护城河。

13.2 Stage 1: WHY 提取

子代理：model: Opus（需要深度推理） **输入**：积木(≤2500) + 代码片段(≤20K) + repo_facts.json **输出**：WHY 卡片（Rationale 对象 + Failure 对象）

每条 WHY 必须附推理链（从代码出发推理）+ 证据（至少一条 CODE 或 DOC）。特别关注项目偏离框架标准的地方。

13.3 Stage 2–3: 概念/规则提取

子代理：model: Sonnet + 积木锚点 **输入**：积木锚点(≤1000) + 代码片段(≤15K) **输出**：概念卡（Capability + Interface 对象）+ 规则卡（Constraint 对象）

规则卡必须带因果注释（`causal_annotation`）——v0.8→v0.9 降分根因就是规则抽离后丢失因果上下文。

13.4 类型化输出持久化（v4 新增）

Stage 1–3 的输出按 6 类知识对象持久化存储，不再仅作为 Stage 4 的临时中间产物。

14. Stage 3.5: 验证关卡

> 确定性脚本执行。LLM 不参与裁决。LLM 是检察官（提出主张 + 附证据），法官是确定性规则引擎。

14.1 事实验证

所有卡片中引用的事实必须在 repo_facts.json 白名单中存在。

14.2 证据链裁决系统

证据类型：`[CODE]` / `[DOC]` / `[COMMUNITY]` / `[INFERENCE]`

裁决规则（版本化，可迭代）：

```
policy_v1:
  R1: ≥1 CODE 或 ≥1 DOC 支持，无高优先级冲突 → SUPPORTED
  R2: 仅 COMMUNITY 支持，独立来源 ≥2 → WEAK
  R3: 仅 INFERENCE → WEAK + QUARANTINE
  R4: CODE 与 DOC 直接矛盾 → CONTESTED
  R5: 可执行检查反证 → REJECTED
```

来源优先级：`runtime CODE > static CODE > DOC > COMMUNITY > INFERENCE`

14.3 项目规模适配

- **高 Star**：强制仓库内证据闭环，LLM 预训练记忆只做候选不入库
- **低 Star**：允许更多 INFERENCE 进 ALLOW_STORY（不进 ALLOW_CORE）

15. Knowledge Compiler

> 确定性编译。不增删内容，只做格式转换。

15.1 按知识类型路由格式

知识类型	编译格式	理由
事实型	结构化列表	精确引用
规则型	规则语句 + 因果注释	保留因果上下文
哲学型	纯叙事	LLM 最擅长理解叙事
踩坑型	案例故事	经验知识用故事最有效

15.2 按证据裁决过滤

裁决	处理
SUPPORTED	纳入核心
WEAK	降为注释，标注"未确认"
CONTESTED	保留冲突双方

裁决	处理
QUARANTINE	不纳入，存档待验证
REJECTED	丢弃

15.3 输出排列

按 Lost in the Middle U 型注意力分布：开头放哲学+关键规则，中间放踩坑故事，尾部放能力清单+速查表。总 token 预算 ≤2000。

15.4 编译目标扩展（v4 新增）

从「一份 CLAUDE.md」扩展为「类型化对象集 + 可选叙事合成」： – 整份抄作业 → 编译为叙事文档（SKILL.md / CLAUDE.md） – 拼零件 → 保持类型化对象，支持跨项目匹配

15.5 风险降级

实验验证（Knowledge Compiler A/B 测试）：有编译 vs 无编译 = 14 vs 14 平局，但编译格式省 43% tokens。结论：不是技术攻关问题，是一天的工程量。风险从"高"降为"低"。

16. Stage 4: 叙事合成（可选）

子代理：model: Opus **输入**：编译产物(≤5K) + SKILL.md 模板(≤500)，无项目代码 **输出**：完整 SKILL.md

v4 变更：Stage 4 从必选变可选。整份抄作业时执行，拼零件时跳过。

17. 灵魂积木系统

17.1 积木规格

层级	范围	Token 预算	示例
L1 框架级	框架核心哲学	200–400 tokens	django-core
L2 模式级	特定模式设计意图	400–800 tokens	django-orm

总注入预算：≤2500 tokens

17.2 积木的三重作用

1. **锚定**：框架公共知识已由积木覆盖，跳过不提取
2. **过滤**：偏离框架核心设计太远的提取结果大概率是噪音
3. **偏离检测基准线**：积木定义"标准做法"，偏差 = WHY 提取的黄金区域

17.3 积木仓库结构

有结构的图，非扁平列表。边类型：`contains` / `composes_with` / `substitutes` / `conflicts_with`。

冷启动优先级：A 类（django, react, express）→ B 类（celery, redis）→ C 类（长尾，不做积木）。

保鲜期 TTL：React hooks ~6 个月；Unix 哲学 ~永不过期。

反积木：记录"千万别这么做"。检测到反模式时重点提取。

17.4 积木确认偏误缓解

风险存在。缓解方案：加负向约束"找出项目偏离框架标准的地方"。具体平衡点留给实验。

18. 子代理架构

18.1 模型路由

阶段	模型	理由
编排/调度	无（确定性）	关键步骤不依赖模型
需求对话	Sonnet	对话理解足够
源泉发现	Sonnet	搜索筛选不需深度推理
WHY/UNSAID 提取	Opus	需要深度推理
概念/规则提取	Sonnet + 积木	有积木辅助足够
叙事合成	Opus	高质量叙事需深度语言能力
冒烟测试	Sonnet	验证任务不需深度推理

模型能力抽象层（v4 新增）：管线逻辑绑定任务类型（"WHY 提取需要深度推理"），不绑定模型品牌。实现路径：MVP 配置文件 → 能力探测 → 动态降级。

18.2 上下文预算

每个子代理独立上下文，不互相挤压（这是调度问题，不是预算问题）：

阶段	内容	预算
Stage 1	积木(≤2500) + 代码(≤20K)	~23K
Stage 2–3	积木锚点(≤1000) + 代码(≤15K)	~16K
Stage 4	编译产物(≤5K) + 模板(≤500)	~6K

18.3 降级策略

子代理调度失败 → 自动降级为单代理串行模式（v1.0 管线作为 fallback）。

18.4 OpenClaw 适配

约束	适配策略
SOUL.md 加载不可靠	关键知识写入 SKILL.md content 区
子代理不继承 SOUL.md	指令通过 AGENTS.md 显式传入
默认 MiniMax（能力有限）	关键阶段显式指定 Opus/Sonnet
异步执行	确定性检查点 + 质量门控
最多 5 并发	Stage 1/2/3 可并行，Stage 4 串行

19. 影子锻造

19.1 出生证明

```
doramagic_origin:
  tool_id: "stock-tracker-2026-abc"
  forge_timestamp: "2026-03-13T10:00:00Z"
  engine_version: "v3.0.5"
  compiler_version: "v1.2.0"
  bricks_used:
    - {id: "akshare-core", version: "v1.0.0"}
  source_repos:
```

```
    - {url: "github.com/akfamily/akshare", commit: "abc123"}
  user_intent_snapshot: "A股实时行情追踪"
  customization_hash: "h123456"
```

19.2 更新流程

用户触发 → 扫描带 doramagic_origin 的 Skill → 比对版本 → 后台影子锻造 → 推送确认 → 保留 MEMORY.md → 支持一键回滚。

Part V: 质量保证体系

---

20. 四层门控

门控	阶段	失败处理
需求完整性	Phase 1 后	继续对话
源泉充分性	Phase 2 后	回退补充搜索
提取质量	Phase 3.5	换源泉或重试
道具可用性	Phase 5	回退修复

21. 提取噪音控制

噪音类型	解法
幻觉（错误信号）	三层防护：第一性原理约束 + 社区交叉验证 + 项目规模调节
不相关事实	积木减少搜索面 + 热力图指导优先级
冗余信息	Knowledge Compiler 去重压缩
过度解释	token 预算硬约束

"不相关但正确"是最大噪音源：当前全量提取事后筛选，缺乏提取阶段的相关性过滤。积木的双重作用（锚定 + 过滤）是主要解法。

22. 幻觉三层防护

1. **第一性原理约束（提取时）**：逼模型从代码出发推理，每条 WHY 附推理链
2. **社区证据交叉验证（验证时）**：用 Issues/RFC/博客找支持/反驳证据
3. **项目规模调节策略（元策略）**：高 Star 警惕记忆污染，低 Star 更依赖代码推理链

23. 模块化验证原则

组件	独立验证方法
repo_facts 提取器	对照人工标注 ground truth
积木质量	专家审核 + A/B 对比
WHY 提取	人工评估推理链质量 + 幻觉率
证据裁决引擎	删证据单调性测试 + 注入伪证据鲁棒性测试
Knowledge Compiler	有/无编译 A/B 对比
最终道具	端到端任务成功率 + 用户纠错次数

Part VI: 实验成果与开发路线

---

24. 实验历史

版本	实验	模型	关键成果
v0.3	exp01	Opus	首次验证：42%→96%（无灵魂→完整灵魂）
v0.5	exp02	MiniMax	首次 MiniMax 全 Stage 通过

版本	实验	模型	关键成果
v0.8	exp05	Superpowers	专家叙事范式验证
v0.9	exp06	Superpowers	三路分离 + 跨模型 benchmark
v0.9 UT	exp07	MiniMax	Telegram 端到端 + 幻觉根因（50% 严重回归）
v0.9+P0	exp08	MiniMax	5 个 P0 补丁 + wger 适应性测试（traceability 100%）

25. WHY 提取实验结论

3 个项目 × 2 个模型 × 3 个版本 = 18 组对照数据。

核心结论： 1. **输入质量是天花板**：funNLP +44% 提升来自 Stage 0 改进 2. **两个杠杆独立且可叠加**：Prompt 优化 +0.5 + Stage 0 优化 +0.7，叠加 +20% 3. **Stage 0 v2 触发全新推理路径**：不是"更多→更好"，是看到完全不同的维度 4. **Prompt 优化已达边际递减**：下一量级需要 Agentic 提取 5. **跨模型：Sonnet > Gemini，但互补**：高分 WHY 几乎不重叠

正式开发投资优先级：

排序	投资	ROI
1	Agentic 提取（Agent 动态读代码）	★★★★★
2	多轮去重（传入已有 WHY 列表）	★★★★
3	GDS 多模型融合 pipeline	★★★
4	actionable_insight 字段	★★★

26. 管线风险评估

三个高风险单点形成**风险三角**：

组件	难度	风险	理由
WHY 提取（Stage 1）	高	高	核心护城河，最容易幻觉
证据链裁决（Stage 3.5）	高	高	全新系统，校准极难

组件	难度	风险	理由
~~Knowledge Compiler~~	~~高~~	**低**	A/B 实验证明是工程量不是技术问题

27. 开发路线图

27.1 exp09 前置任务

#	任务	目的	状态
1	WHY 分类框架 + 评判 rubric	实验有标尺	进行中
2	2–3 个项目手工 WHY 答案卷	实验有 ground truth	未开始
3	v1.0 输出系统复盘	已知失败模式指导 prompt	未开始
4	django-core + django-orm 积木	验证积木增益假设	未开始

27.2 MVP 路径

```
Step 1: 积木冷启动 → django-core + django-orm → 验证积木增益
Step 2: Soul Extractor v2 → 确定性编排 + 子代理 + 证据裁决
Step 3: 端到端验证 → "帮我做 Django 管理后台" → 可用 Skill
Step 4: 跨领域验证 → Django → 金融/健身
```

27.3 需在实验中验证的假设

假设	验证方法	影响
积木能提升提取质量	有/无积木 A/B	积木系统是否需要重设计
证据链裁决比 A/B/C/D 更准	对比实验	否则退回 A/B/C/D
子代理架构比单代理好	对比实验	否则简化为单代理
热力图维度有区分度	3 项目验证	决定保留哪些维度
Constraint 显式提取有效	exp09	6 类知识对象体系是否成立

附录

附录 A: 硬性约束清单

1. **第一性原理**：所有技术决策从根本事实推导
2. **产品设计之魂**：不教用户做事，给他工具
3. **代码说事实，AI 说故事**：事实层 AI 不许编，解读层 AI 可以解读但不能瞎编
4. **OpenClaw 适配**：Doramagic 适应平台，不可反过来
5. **Knowledge Compiler 不增删内容**：只做格式转换
6. **能力显性**：Doramagic 的能力让用户看得见
7. **完整交付**：永远交付完整的工作成果，scope 随覆盖度变化（v4 新增）
8. **不自我矮化**：不叫自己"半成品"，不虚假完整（v4 新增）

附录 B: 接口契约

Stage 0 → Stage 1/2/3

```
repo_context:
  repo_facts: RepoFacts
  framework_id: string
  matched_bricks: BrickRef[]
  heatmap: HeatmapEntry[]
  project_narrative: string          # v4 新增
  code_snippets:
    - path: string
      content: string
      heat_label: string
```

Stage 1/2/3 → Stage 3.5

```
knowledge_card:
  claim_id: string
  layer: enum[domain|operation|experience|decision|philosophy|implicit]
  object_type: enum[capability|rationale|constraint|interface|failure]  # v4 新增
  text: string
  reasoning_chain: string
  causal_annotation: string
```

```
  evidence_profile:
    - evidence_id: string
      type: enum[CODE|DOC|COMMUNITY|INFERENCE]
      source: string
      reproducible: boolean
```

Stage 3.5 → Knowledge Compiler

```
judged_card:
  card: KnowledgeCard
  verdict: enum[SUPPORTED|CONTESTED|WEAK|REJECTED]
  policy_action: enum[ALLOW_CORE|ALLOW_STORY|QUARANTINE]
  policy_version: string
  conflicts: ConflictRecord[]
```

Knowledge Compiler → Stage 4 / 跨项目匹配

```
compiled_knowledge:
  # 叙事路径（Stage 4 用）
  philosophy_section: string
  rules_section: string
  traps_section: string
  capabilities_section: string
  quickref_section: string
  total_tokens: int
  evidence_footnotes: string[]
  # 类型化路径（跨项目匹配用，v4 新增）
  typed_objects:
    capabilities: Capability[]
    rationales: Rationale[]
    constraints: Constraint[]
    interfaces: Interface[]
    failures: Failure[]
```

附录 C: 5 大技术难点全部闭环

难点	解决方案
1. 幻觉与证据绑定	三层防护 + 证据链裁决
2. 确定性与生成式边界	Knowledge Compiler + 按类型路由

难点	解决方案
3. 跨模型/Agent 控制	确定性编排 + 子代理路由 + 降级
4. 知识密度与可用性	本质是提取噪音问题，积木过滤 + 子代理调度
5. 多源融合与端到端	证据链 + 标签脉络 + 集成

附录 D: 审查结论索引

问题	结论
1. 合理 vs 已验证	模块化验证，验证策略是架构质量试金石
2. 信息漏斗	核心是拉高作业下限与上限
3. 降级策略	子代理失败 → 单代理串行 fallback
4. OpenClaw First	阶段性最优，核心引擎平台无关
5. 积木确认偏误	加负向约束"找偏离"
6. 置信度体系	证据链 + 确定性裁决（三方共识）
7. Token 预算	可配置，留给实验
8. 合规法律	初期不管
9. 多道具冲突	后续解决
10. /trace 调试	影子锻造解决更新

附录 E: 三方研究索引

研究	路径	核心结论
灵魂积木粒度	research/soul-lego-bricks/reports/	L1(200-400) + L2(400-800)，总≤2500

研究	路径	核心结论
AI 知识消费格式	research/ai-knowledge-consumption/reports/	结构化抽取，叙事化编译，分块式注入
置信度体系	research/product-definition-v3/confidence-system-*.md	证据链 + 确定性裁决
影子锻造	research/product-definition-v3/tool-update-mechanism-gemini.md	出生证明 + 影子锻造
热力图	research/product-definition-v3/heatmap-brainstorm-*.md	四种热度，待验证
部分覆盖	docs/research/20260314_doramagic_partial_coverage_super_report.md	Gap-Aware Assembly + Incomplete Skill
部分覆盖 (Gemini)	~/research/Doramagic_Architecture_Research_2026.md	5 个策略隐喻

附录 F: GLM5 可复用成果

1. 能力积木 YAML Schema（四类型 Trigger/Action/Resource/Dependency）
2. 置信度评分模型（四维度加权，70% 阈值）
3. 自适应交互深度（三级用户分层，最多 3 轮）
4. SECI 知识转化模型（六类转化中丢失的信息）
5. 噪音相对性理论（AAOCC 评估框架）
6. 三层验证体系（语法/流程/运行）
7. 进度可视化设计
8. 知识卡片格式（YAML frontmatter + 五类卡片 + E1-E4 证据等级）
9. Soul Extractor vs Doramagic 分工
10. OpenClaw Skill 结构实证（10 个真实 Skill 分析）
11. 灵魂分解研究（Louvain/Leiden 社区检测 + Token 预算模型 + 4 维耦合公式）

附录 G: 竞品参考

- **Google Always-On Memory Agent**：ConsolidateAgent ≈ Knowledge Compiler，验证"先提取再编译"架构。UNSAID 评分机制可借鉴。
- **CodeScene**：代码变更频率×复杂度热力图
- **Structurizr**：C4 Model + ADR 可视化
- **Sourcegraph**：代码引用/依赖关系可视化

> **理论研究阶段全部完成。下一步是 exp09 实验验证。**